چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۴

کوروتین‌ها و هم‌روندی در Kotlin — هم‌روندی ساختاریافته، لغو و امنیت ترد



اسماعیل مرادی

@esmaeeil-moradi700

چرا کوروتین‌ها روز چهارم‌اند؟

همهٔ APIهای مدرن اندروید ناهمگام‌اند — و در ۲۰۲۶ زبانشان `suspend` است. مصاحبه‌گرها دیگر کمتر می‌پرسند «*coroutine* چیست؟»؛ می‌پرسند وقتی **fail می‌شود، لغو می‌شود یا وارد race می‌شود** چه اتفاقی می‌افتد:

- Room، DataStore، Paging، Ktor، Retrofit و WorkManager با `CoroutineWorker` همه API از نوع `suspend` و `Flow` دارند
- **هم‌روندی ساختاریافته (structured concurrency)** جای مدیریت دستی ترد و callback را گرفته — نشت حافظه از اساس منتفی است
- **main-safety** یک قرارداد است: هر تابع suspend را باید بتوان با خیال راحت از ترد اصلی صدا زد
- cancellation بلعیده‌شده، exceptionهای گم‌شده و data race فیلترهای کلاسیک مصاحبهٔ سینیورند
- `kotlinx.coroutines` چندسکویی است — همین کد در ماژول‌های KMP هم اجرا می‌شود

سطح	سؤال‌ها
جونیور	suspend در برابر blocking، dispatcherها و main-safety، تفاوت launch و async
مید	scope و context، scopeهای اندروید، لغو، exceptionها
سینیور	coroutineScope یا supervisorScope، کار موازی با سقف، تست
آرکیتکت	استراتژی خطا، state مشترک قابل‌تغییر، زیر کاپوت



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: درختی از کوروتین‌ها

```
viewModelScope   SupervisorJob + Dispatchers.Main.immediate
 ├── launch { loadProfile() }
 │     └── coroutineScope
 │           ├── async { api.user() }
 │           └── async { api.feed() }
 └── launch { observeCart() }
```

- **لغو به پایین می‌رود:** یک scope را لغو کنید، همهٔ coroutineهای زیرش لغو می‌شوند
- **خطا به بالا می‌رود:** فرزندی که fail شود والد و siblingهایش را لغو می‌کند — مگر اینکه والد supervisor باشد
- **والد منتظر می‌ماند:** یک coroutine فقط وقتی تمام می‌شود که همهٔ فرزندانش تمام شده باشند

سؤال واحد پشت هر جواب دربارهٔ کوروتین

والد این coroutine کیست؟ اگر نمی‌توانید scopeای را نام ببرید که آن را لغو خواهد کرد، یعنی نشت دارید.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

suspend در برابر blocking، dispatcherها و main-safety، تفاوت launch و async



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. suspend کردن در برابر block کردن

	suspend	blocking
ترد در زمان انتظار	آزاد می‌شود و کار دیگری انجام می‌دهد	تا برگشتن فراخوانی گیر می‌کند
مثال‌ها	`delay()`، فراخوانی‌های `suspend` در Room و Retrofit، `withContext`	`Thread.sleep()`، `InputStream.read()`، `runBlocking`
روی ترد اصلی	UI روان می‌ماند	اول jank، بعد ANR (روز ۲)
لغو	در نقطه‌های suspend واکنش نشان می‌دهد	لغو coroutine را نادیده می‌گیرد

coroutine یک **محاسبهٔ قابل‌تعلیق** است: در هر نقطهٔ suspend می‌تواند مکث کند، تردش را پس بدهد و بعداً ادامه دهد — شاید روی یک ترد دیگر.

تلهٔ مصاحبه

«`runBlocking` *در کد production مشکلی ندارد.*» — در اندروید، تردی را که رویش اجرا می‌شود block می‌کند؛ روی ترد اصلی یعنی UI یخ‌زده. جایش در تابع `main()` و چند مورد خاص برای پل زدن است — نه در کد اپ، و نه در تست (از `runTest` استفاده کنید).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. `suspend` واقعاً یعنی چه

- `suspend` یعنی *«این تابع ممکن است بدون block کردن مکث کند»*. فقط از داخل یک coroutine یا یک تابع suspend دیگر صدا زده می‌شود.
- ترد را **انتخاب نمی‌کند**. بدنهٔ تابع روی dispatcher صدازننده اجرا می‌شود، تا وقتی که چیزی در داخلش با `withContext` جابه‌جا شود.

```
// Called from viewModelScope → runs on the main thread
suspend fun loadAvatar(url: String): Bitmap {
    val bytes = api.download(url)   // suspends: thread is free
    // CPU work: still on the caller's thread (main!)
    return BitmapFactory.decodeByteArray(bytes, 0, bytes.size)
}
```

تلهٔ مصاحبه

«تابعی را که `suspend` علامت بزنید از ترد اصلی خارج می‌شود.» — **نه.** تابع suspend که decode یا parse می‌کند یا I/O مسدودکننده صدا می‌زند، همچنان ترد صدازننده را block می‌کند. داخل خود تابع درستش کنید: `withContext(Dispatchers.Default) { … }`.

تلهٔ مصاحبه

«coroutine یعنی ترد سبک.» — تشبیه مفیدی است، اما مدل ذهنی غلطی است. coroutine کاری است که **روی** تردها زمان‌بندی می‌شود؛ بعد از هر suspend ممکن است روی ترد دیگری از همان dispatcher ادامه پیدا کند، پس مقدارهای `ThreadLocal` همراهش نمی‌آیند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. dispatcherها — کد شما روی کدام ترد اجرا می‌شود

dispatcher	کاربرد	نکتهٔ مهم
`Main`	UI، و صدا زدن تابع‌های suspend امن برای ترد اصلی	همان ترد اصلی اندروید
`Main.immediate`	همان، ولی اگر از قبل روی ترد اصلی باشید دوباره dispatch نمی‌کند	پیش‌فرض `viewModelScope` و `lifecycleScope`
`Default`	کار CPU: مرتب‌سازی، parse، diff، پردازش تصویر	به تعداد هسته‌های CPU ترد دارد (حداقل ۲)
`IO`	I/O مسدودکننده: فایل، SDKهای قدیمی blocking	تا ۶۴ ترد (روی دستگاه‌های با هستهٔ بیشتر، بیشتر)؛ تردها را با `Default` شریک است
`Unconfined`	تقریباً هرگز در کد اپ	روی هر تردی که resumeاش کند ادامه می‌دهد

تلهٔ مصاحبه

«`Dispatchers.Default` *همان* `ForkJoinPool` *است.*» — در نسخه‌های مدرن kotlinx.coroutines نه (نسخهٔ پارسال همین چالش این را گفته بود). `Default` و `IO` یک زمان‌بند مشترک دارند؛ جابه‌جایی بین این دو اغلب حتی ترد را هم عوض نمی‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. main-safety در عمل

```
class ReportRepository(
    private val dao: ReportDao,
    private val cpu: CoroutineDispatcher = Dispatchers.Default,
    private val io: CoroutineDispatcher = Dispatchers.IO,
) {
    // Main-safe: callers never switch threads
    suspend fun export(file: File) {
        val rows = dao.getAll()   // Room suspend: main-safe
        val csv = withContext(cpu) { rows.toCsv() }
        withContext(io) { file.writeText(csv) } // blocking I/O
    }
}
```

- تابعی که کار سنگین را انجام می‌دهد مسئول `withContext` است — نه صدازننده‌اش
- dispatcherها را **inject کنید** تا تست بتواند جایگزینشان کند (سؤال ۱۰)

تلهٔ مصاحبه

«*هر فراخوانی Room یا Retrofit را داخل `withContext(Dispatchers.IO)` بگذارید.*» — تابع‌های `suspend` آن‌ها خودشان main-safe هستند. این جابه‌جایی اضافه چیزی اضافه نمی‌کند؛ `withContext` را برای کد مسدودکننده یا سنگین CPU خودتان نگه دارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. تفاوت launch و async

	launch	async
خروجی	`Job`	`Deferred<T>` — یک `Job` همراه با نتیجه
نتیجه	ندارد — فقط side effect	`await()` تا آماده شدن نتیجه **suspend** می‌کند
خطا	به والد منتقل می‌شود	این هم به والد منتقل می‌شود؛ `await()` دوباره throwاش می‌کند
کاربرد	ذخیره، refresh، شروع collect	چند نتیجهٔ مستقل به‌صورت موازی

```
suspend fun loadDashboard(): Dashboard = coroutineScope {
    val user = async { api.user() }    // both start now
    val feed = async { api.feed() }
    Dashboard(user.await(), feed.await())
}   // total time ≈ the slower call, not the sum
```

اگر یکی از دو فراخوانی fail شود، دیگری لغو می‌شود و exception از `loadDashboard()` دوباره throw می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*خطای `async` تا وقتی کسی `await()` را صدا نزند پنهان می‌ماند.*» — غلط است، و نسخهٔ پارسال همین را گفته بود. یک `async` **فرزند** که fail شود، والد و siblingهایش را فوراً لغو می‌کند. فقط `async` **ریشه** — مثلاً `viewModelScope.async { }` — exception را داخل `Deferred` نگه می‌دارد؛ و اگر کسی awaitاش نکند، خطا بی‌صدا گم می‌شود.

تلهٔ مصاحبه

«`await()` *تا آماده شدن نتیجه block می‌کند.*» — **suspend** می‌کند. ترد به کارهای دیگرش ادامه می‌دهد.

سؤال تکمیلی

«`async { repo.load() }.await()` *چه ایرادی دارد؟*» — شروع کردن و بلافاصله await کردن، همان کد ترتیبی است با هزینهٔ اضافه. مستقیم `repo.load()` را صدا بزنید، یا اگر dispatcher دیگری لازم دارید از `withContext` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

scope و context، scopeهای اندروید، لغو، exceptionها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. CoroutineScope و CoroutineContext

```
val scope = CoroutineScope(
    SupervisorJob() + Dispatchers.Default +
        CoroutineName("sync")
)

scope.launch(Dispatchers.IO) {
    // context = scope's context + IO (overrides Default)
    //           + a new Job, child of the scope's Job
}

scope.cancel()  // cancels every coroutine started in it
```

- **context** مجموعه‌ای از عنصرهاست، از هر کلید یکی: `Job` ، dispatcher، `CoroutineName` ، `CoroutineExceptionHandler`
- عملگر `+` دو context را ادغام می‌کند؛ سمت راست برنده است
- **scope** فقط نگه‌دارندهٔ یک context است — `Job` آن والد هر چیزی است که در آن launch می‌شود

سؤال تکمیلی

«فرق *`coroutineScope { }`* با *`CoroutineScope(…)`* چیست؟» — `coroutineScope` یک تابع suspend است: scope فرزندی که منتظر فرزندانش می‌ماند و خطایشان را دوباره throw می‌کند. `CoroutineScope(…)` یک scope **ریشهٔ** جدید می‌سازد که لغوش با خود شماست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. شکستن ساختار، بی‌آنکه بفهمید

```
// ❌ A new Job becomes the parent: escapes scope.cancel()
scope.launch { launch(Job()) { upload() } }

// ❌ Same problem, and the children are NOT supervised
withContext(SupervisorJob()) { uploadAll() }

// ❌ No owner at all: can't be cancelled or replaced in tests
GlobalScope.launch { upload() }
```

تلهٔ مصاحبه

«`launch(SupervisorJob())` *این فرزند را در برابر خطا امن می‌کند.*» — نمی‌کند. پاس دادن یک `Job` به builder آن را **والد جدید** می‌کند: coroutine از scope خودش جدا می‌شود، همراه آن لغو نمی‌شود، و فرزندان خودش هم باز با هم fail می‌شوند. از `supervisorScope { }` استفاده کنید (سؤال ۸).

قاعدهٔ سرانگشتی

به `launch` و `withContext` می‌توانید dispatcher، نام و handler بدهید — اما هرگز `Job` .



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. scopeهای اندروید — چه کسی چه چیزی را لغو می‌کند

scope	کِی لغو می‌شود	کاربرد
`viewModelScope`	وقتی ViewModel پاک شود	منطق صفحه: بارگذاری، refresh، ذخیره
`lifecycleScope`	وقتی lifecycle به DESTROYED برسد	کار یک‌بارهٔ UI در Activity و Fragment
`repeatOnLifecycle(STARTED)`	در STOP لغو و در START دوباره شروع می‌شود	collect کردن Flow در Viewها (روز ۵)
`rememberCoroutineScope` و `LaunchedEffect`	وقتی از composition خارج شود	event handlerها و effectهای Compose (روز ۸)
scope سطح اپ که inject شده	هرگز — هم‌عمر process است	کاری که باید بعد از رفتن کاربر هم تمام شود
WorkManager	از process death هم عبور می‌کند	کار تضمین‌شده (روز ۱۵)

تلهٔ مصاحبه

«*`lifecycleScope` در `onStop` لغو می‌شود.*» — نه، فقط در DESTROYED (نسخهٔ پارسال این را اشتباه گفته بود). collectorای که با `lifecycleScope.launch` شروع شود در background هم ادامه می‌دهد — دلیل وجود `repeatOnLifecycle` همین است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. کاری که باید از صفحه بیشتر عمر کند

```
class DraftRepository(
    private val api: DraftApi,
    private val appScope: CoroutineScope, // injected, app-wide
) {
    suspend fun publish(draft: Draft) {
        // Survives the user leaving the screen
        appScope.launch { api.publish(draft) }.join()
    }
}
```

صدازننده همچنان با `join()` منتظر می‌ماند. اگر ViewModel پاک شود، فقط *انتظار* لغو می‌شود — انتشار ادامه پیدا می‌کند.

تلهٔ مصاحبه

«*برای متوقف کردن کار در background از* `launchWhenStarted` *استفاده کنید.*» — deprecated شده است. فقط coroutine را **suspend** می‌کند و producerهای بالادستی همچنان کار می‌کنند. `repeatOnLifecycle` به‌جایش لغو می‌کند و دوباره شروع می‌کند.

تلهٔ مصاحبه

«*خب از* `GlobalScope` *استفاده کنید.*» — hard-code شده است، نمی‌شود گروهی لغوش کرد و در تست قابل‌جایگزینی نیست. به‌جایش یک scope سطح اپ inject کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. لغو، مشارکتی است

`cancel()` یک coroutine را متوقف نمی‌کند — از آن **می‌خواهد** که متوقف شود. coroutine در نقطهٔ suspend یا چک بعدی متوجه می‌شود.

```
suspend fun compress(files: List<File>) = withContext(cpu) {
    for (file in files) {
        ensureActive()       // throws if cancelled
        compressOne(file)    // CPU-heavy, never suspends
    }
}

// A blocking call that should react to cancellation
suspend fun readLegacy(): Data =
    runInterruptible(Dispatchers.IO) {
        legacySdk.readBlocking()
    }
```

- تابع‌های suspend در `kotlinx.coroutines`، مثل `delay`، `withContext`، `await` و `join`، قابل‌لغو هستند
- حلقه‌های خالص CPU و فراخوانی‌های مسدودکننده **نیستند** — `ensureActive()` یا `yield()` اضافه کنید، یا داخل `runInterruptible` بگذاریدشان



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. پاک‌سازی، timeout — و باگ کلاسیک

```
scope.launch {
    try {
        upload(file)
    } catch (e: IOException) {
        showError(e)
    } finally {
        withContext(NonCancellable) {
            dao.markUploadIdle()   // suspend call in cleanup
        }
    }
}

val result = withTimeoutOrNull(10.seconds) { api.fetch() }
```

تلهٔ مصاحبه

`catch (e: Exception)` اکسپشن `CancellationException` را هم می‌گیرد — و coroutine بعد از لغو شدن به کارش ادامه می‌دهد. نسخهٔ پارسال گفته بود «بگیریدش و فقط log کنید». همیشه دوباره **throwاش کنید**؛ همین برای `runCatching` هم صادق است (روز ۳).

سؤال تکمیلی

«چرا *من بی‌صدا شکست خورد؟*» — `withTimeout` خودش یک `TimeoutCancellationException` است: داخل `launch` فقط coroutine را تمام می‌کند — نه crash، نه handler. از `withTimeoutOrNull` استفاده کنید و حالت `null` را مدیریت کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. exceptionها — به کجا می‌روند

```
// ❌ Never catches: it's thrown inside the coroutine
try {
    scope.launch { error("boom") }
} catch (e: Exception) { }

// ✅ Handle expected failures inside the coroutine
scope.launch {
    try {
        repo.refresh()
    } catch (e: IOException) {
        showError(e)
    }
}

// ✅ Last resort: log on the root coroutine
val handler = CoroutineExceptionHandler { _, e -> log(e) }
viewModelScope.launch(handler) { repo.refresh() }
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. قاعده‌ها

- **در `launch`:** exception گرفته‌نشده **به بالا**، به والد می‌رود. در ریشه به `CoroutineExceptionHandler` می‌رسد — یا به uncaught handler ترد، که در اندروید اپ را **crash** می‌کند.
- **در `async`:** تابع `await()` exception را دوباره throw می‌کند — و `async` فرزند باز هم والدش را fail می‌کند (سؤال ۳).
- `CoroutineExceptionHandler` فقط روی coroutine **ریشه** یا scope آن کار می‌کند.
- **بعد از** fail شدن coroutine اجرا می‌شود — برای log کردن است، نه بازیابی.

تلهٔ مصاحبه

«handler را روی همان فرزندی بگذارید که ممکن است fail شود.» — `launch(handler)` داخل یک coroutine دیگر **نادیده گرفته می‌شود:** exception باز هم به والد می‌رود و handler والد (اگر باشد) آن را دریافت می‌کند.

سؤال تکمیلی

«آیا `CoroutineExceptionHandler` می‌تواند بعد از یک باگ اپ را زنده نگه دارد؟» — از نظر فنی جلوی crash را می‌گیرد؛ از نظر معماری اشتباه است. سؤال ۱۱ را ببینید.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

coroutineScope یا supervisorScope، کار موازی با سقف، تست کوروتین‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. coroutineScope یا supervisorScope

	`coroutineScope` و `Job`	`supervisorScope` و `SupervisorJob`
وقتی یک فرزند fail شود	siblingها لغو و خطا دوباره throw می‌شود	فقط همان فرزند fail می‌شود
مدل	همه یا هیچ	کارهای مستقل
exception گرفته‌نشدهٔ فرزند	به والد می‌رود	به handler خود فرزند می‌رود — یا crash
مثال	پروفایل به‌همراه دسترسی‌هایش: جدا از هم بی‌فایده‌اند	صفحهٔ خانه: اخبار، آب‌وهوا، پیشنهادها

`viewModelScope` و `lifecycleScope` از `SupervisorJob` استفاده می‌کنند: یک `launch` ناموفق بقیه را لغو نمی‌کند.

تلهٔ مصاحبه

supervision فقط برای **فرزندان مستقیم** است. داخل یک فرزند supervised، نوه‌هایی که با `launch` معمولی شروع شده‌اند باز هم با هم fail می‌شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. صفحه‌ای که شکست جزئی را تحمل می‌کند

```
suspend fun loadHome(): Home = supervisorScope {
    val news = async { api.news() }
    val weather = async { api.weather() }
    Home(news.awaitOrNull(), weather.awaitOrNull())
}

suspend fun <T> Deferred<T>.awaitOrNull(): T? = try {
    await()
} catch (e: CancellationException) {
    throw e
} catch (e: IOException) {
    null
}
```

سرویس آب‌وهوا از کار افتاده؟ صفحهٔ خانه همچنان اخبار را نشان می‌دهد.

تلهٔ مصاحبه

«`supervisorScope` یعنی *crash نداریم.*» — فقط جلوی پخش شدن خطا به siblingها را می‌گیرد. exception مدیریت‌نشده در یک `launch` داخل supervisor همچنان به uncaught handler می‌رسد و اپ را **crash** می‌کند. داخل هر فرزند catch کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. کار موازی با سقف مشخص

«۲۰۰ عکس آپلود کن — حداکثر ۴ تا هم‌زمان، هر کدام ۳۰ ثانیه، و اگر کاربر صفحه را ترک کرد همه لغو شوند.»

```
suspend fun uploadAll(photos: List<Photo>) = coroutineScope {
    val permits = Semaphore(4)
    photos.map { photo ->
        async {
            permits.withPermit {
                withTimeout(30.seconds) { api.upload(photo) }
            }
        }
    }.awaitAll()
}
```

- `Semaphore` هم‌زمانی را محدود می‌کند؛ بقیهٔ coroutineها در انتظار مجوز **suspend** می‌شوند — هیچ تردی block نمی‌شود
- `coroutineScope` هر ۲۰۰ تا را به صدازننده گره می‌زند: صدازننده لغو شود، همهٔ آپلودها متوقف می‌شوند
- یک خطا بقیه را لغو می‌کند. موفقیت جزئی می‌خواهید؟ `supervisorScope` به‌همراه یک نتیجه برای هر آیتم

تلهٔ مصاحبه

`photos.forEach { launch { api.upload(it) } }` — یعنی ۲۰۰ درخواست هم‌زمان. coroutine ارزان است؛ کاری که شروع می‌کند نه. و `withTimeout` **بیرون** از `withPermit` زمان انتظار در صف را هم حساب می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. تست کوروتین‌ها — جعبه‌ابزار

ابزار	چه می‌کند
`runTest { }`	تست را در یک `TestScope` اجرا می‌کند؛ `delay` با **زمان مجازی** رد می‌شود
`StandardTestDispatcher`	coroutineهای جدید را در صف نگه می‌دارد تا با `runCurrent()` یا `advanceUntilIdle()` اجرا شوند
`UnconfinedTestDispatcher`	coroutineهای جدید را بلافاصله شروع می‌کند — با همان ساعت مجازی
`advanceTimeBy()` و `currentTime`	جلو بردن و خواندن ساعت مجازی
`backgroundScope`	برای collectorهای بی‌پایان؛ در پایان تست لغو می‌شود
`Dispatchers.setMain()`	جایگزینی `Main` برای `viewModelScope` — همان `MainDispatcherRule` روز ۳

تلهٔ مصاحبه

«*`UnconfinedTestDispatcher` زمان مجازی را نادیده می‌گیرد.*» — غلط است (نسخهٔ پارسال). هر دو dispatcher تست یک `TestCoroutineScheduler` مشترک دارند؛ تنها تفاوت، **زمان شروع** coroutineهای جدید است.

تلهٔ مصاحبه

`runBlockingTest` و `TestCoroutineDispatcher` منسوخ (deprecated) شده‌اند. از `runTest` با dispatcherهای تست بالا استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. زمان مجازی در عمل

```
suspend fun <T> retryWithBackoff(block: suspend () -> T): T {
    var wait = 1.seconds
    repeat(2) {
        try { return block() }
        catch (e: IOException) { delay(wait); wait *= 2 }
    }
    return block()
}

@Test fun retriesWithBackoff() = runTest {
    var calls = 0
    val result = retryWithBackoff {
        if (++calls < 3) throw IOException("offline") else "ok"
    }
    assertEquals("ok", result)
    assertEquals(3, calls)
    assertEquals(3_000L, currentTime) // 1 s + 2 s, virtual
}
```

این تست در چند میلی‌ثانیه تمام می‌شود: `delay` به‌جای خوابیدن، ساعت مجازی را جلو می‌برد.

تلهٔ مصاحبه

کلاسی که `Dispatchers.IO` را hard-code کرده از زمان‌بند تست فرار می‌کند: تأخیرهایش واقعی‌اند و زمان‌بندی‌اش ناپایدار (flaky). dispatcher را **inject کنید** و در تست یک `StandardTestDispatcher(testScheduler)` بدهید.



اسماعیل مرادی
@esmaeeil-moradi700

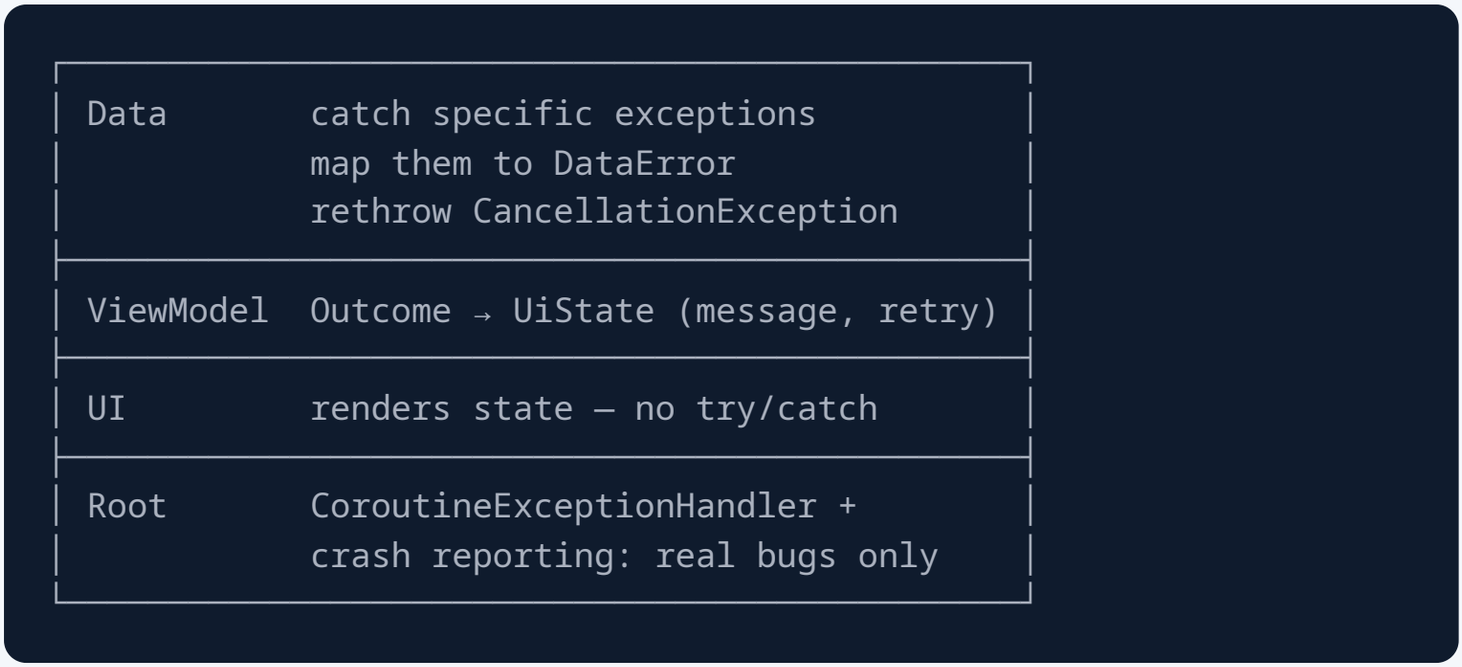
آرکیتکت

استراتژی خطا در لایه‌ها، state مشترک قابل‌تغییر، زیر کاپوت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. یک استراتژی خطا برای کل اپ



- **خطاهای قابل‌انتظار** (آفلاین بودن، خطای ۴۰۱، اعتبارسنجی) داده‌اند: مدلشان کنید، نشانشان بدهید، امکان تلاش دوباره بدهید
- **باگ‌ها** (`NullPointerException`، شکستن invariantها) باید سریع fail شوند و به crash reporting برسند

> **قاعدهٔ سرانگشتی**
>
> exception را فقط در لایه‌ای بگیرید که واقعاً **کاری** برایش از دستش برمی‌آید. بقیه باگ است — بگذارید بالا برود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. خطاهای قابل‌انتظار به شکل مقدار

```
sealed interface DataError {
    data object Network : DataError
    data object Unauthorized : DataError
    data class Server(val code: Int) : DataError
}

suspend fun <T> safeCall(block: suspend () -> T): Outcome<T> =
    try {
        Outcome.Ok(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Outcome.Err(DataError.Network)
    } catch (e: HttpException) {
        Outcome.Err(e.toDataError())
    }
```

`Outcome` یک sealed type کوچک است با دو حالت `Ok(value)` و `Err(error)`. فقط exceptionهایی که **انتظارشان را دارید** گرفته می‌شوند — هر چیز دیگری باگ است و بالا می‌رود.

تلهٔ مصاحبه

«یک `CoroutineExceptionHandler` *سراسری که همه چیز را بگیرد تا اپ هیچ‌وقت crash نکند.*» — نسخهٔ پارسال پیشنهاد داده بود NPEها را این‌طور بگیریم. handler وقتی اجرا می‌شود که coroutine مرده است: صفحه نیمه‌کاره به‌روز شده و باگ‌های واقعی پنهان می‌مانند. خطاهای قابل‌انتظار را همان‌جا که رخ می‌دهند مدیریت کنید؛ بگذارید باگ‌ها با صدای بلند fail شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. state مشترک قابل‌تغییر

```
var count = 0
coroutineScope {
    repeat(1_000) {
        launch(Dispatchers.Default) { repeat(100) { count++ } }
    }
}   // count is usually LESS than 100 000: lost updates
```

`count++` یعنی خواندن، تغییر و نوشتن؛ coroutineهای موازی نتیجهٔ همدیگر را بازنویسی می‌کنند. یک مورد واقعی — token فقط یک بار refresh شود:

```
class TokenProvider(private val api: AuthApi) {
    private val mutex = Mutex()
    private var token: Token? = null

    // One refresh at a time; waiting callers reuse the result
    suspend fun validToken(): Token = mutex.withLock {
        token?.takeIf { !it.isExpired() }
            ?: api.refresh().also { token = it }
    }
}
```

تلهٔ مصاحبه

«خب از *synchronized* استفاده کنید.» — بلوک `synchronized` نمی‌تواند نقطهٔ suspend داشته باشد (کامپایلر ردش می‌کند)، و در زمان انتظار یک ترد را block می‌کند. `Mutex` به‌جایش suspend می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. جعبه‌ابزار

ابزار	کِی	مراقب باشید
`AtomicInteger` و `AtomicReference` در JVM، و atomicfu در KMP	یک مقدار: شمارنده، flag، compare-and-set	فقط یک متغیر در هر لحظه
`MutableStateFlow.update { }`	state قابل‌مشاهده	lambda ممکن است بیش از یک بار اجرا شود — خالص نگهش دارید
`Mutex.withLock { }`	بخش بحرانی چندمرحله‌ای که suspend می‌کند	reentrant نیست: `withLock` تودرتو روی همان mutex به deadlock می‌رسد
`limitedParallelism(1)`	محدود کردن یک منبع غیر thread-safe به یک worker	فقط **بین** نقطه‌های suspend اتمیک است
یک coroutine به‌همراه یک `Channel`	یک مالک واحد که فرمان‌ها را به ترتیب پردازش می‌کند	builder مربوط به `actor { }` منسوخ است — همان که پارسال توصیه شده بود

تلهٔ مصاحبه

«*`limitedParallelism(1)` کد من را اتمیک می‌کند.*» — فقط بین نقطه‌های suspend. کدی مثل `val x = state; delay(…); state = x + 1` باز هم به‌روزرسانی‌ها را گم می‌کند: وقتی این coroutine معلق است، coroutine دیگری اجرا می‌شود. `Mutex` در طول suspend هم نگه داشته می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. زیر کاپوت: CPS و یک state machine

```
// You write:
suspend fun loadUser(id: String): User {
    val token = auth.token()        // suspension point 1
    return api.user(id, token)      // suspension point 2
}

// The compiler emits (simplified):
fun loadUser(id: String, cont: Continuation<User>): Any?
// returns a User — or COROUTINE_SUSPENDED
```

- **سبک CPS یا continuation-passing:** هر تابع suspend یک پارامتر پنهان `Continuation` می‌گیرد — یعنی «ادامهٔ کار»، که با `resumeWith(result)` از سر گرفته می‌شود
- بدنهٔ تابع به یک **state machine** تبدیل می‌شود: یک `label` نگه می‌دارد از کجا ادامه دهد؛ متغیرهای محلی که بعداً لازم‌اند در شیء continuation، روی heap، ذخیره می‌شوند
- اگر `auth.token()` مقدار `COROUTINE_SUSPENDED` برگرداند، `loadUser` هم برمی‌گردد و ترد آزاد می‌شود؛ بعداً `resumeWith` از label بعدی وارد می‌شود

تلهٔ مصاحبه

«*کوروتین‌ها کد را سریع‌تر می‌کنند.*» — **انتظار** را ارزان می‌کنند. کار سنگین CPU سریع‌تر نمی‌شود؛ موازی‌سازی واقعی همچنان به تعداد هسته‌ها محدود است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. continuation در کد خودتان

یک API یک‌باره مبتنی بر callback را به `suspend` تبدیل کنید (اینجا OkHttp):

```
suspend fun Call.await(): Response =
    suspendCancellableCoroutine { cont ->
        cont.invokeOnCancellation { cancel() }  // Call.cancel()
        enqueue(object : Callback {
            override fun onResponse(c: Call, r: Response) =
                cont.resume(r)
            override fun onFailure(c: Call, e: IOException) =
                cont.resumeWithException(e)
        })
    }
```

- برای جریانی از callbackها (موقعیت مکانی، سنسورها) به‌جایش `callbackFlow` لازم است — روز ۵

تلهٔ مصاحبه

استفاده از `suspendCoroutine` به‌جای آن: coroutine در زمان انتظار **قابل‌لغو نیست** و درخواست همچنان اجرا می‌شود. همیشه `suspendCancellableCoroutine` را ترجیح دهید و کار زیرین را لغو کنید.

تلهٔ مصاحبه

«*هزاران ترد باعث* `StackOverflowError` *می‌شوند.*» — نسخهٔ پارسال این را گفته بود. ترد زیاد **حافظه** را تمام می‌کند (ساختن ترد با `OutOfMemoryError` شکست می‌خورد)؛ `StackOverflowError` از بازگشتی (recursion) بیش از حد عمیق می‌آید.



اسماعیل مرادی
@esmaeeil-moradi700

روز ۴ — خلاصهٔ یک‌صفحه‌ای

- `suspend` یعنی «ممکن است بدون block کردن مکث کند». ترد را **انتخاب نمی‌کند**.
- main-safety: تابعی که کار سنگین می‌کند مسئول `withContext` است؛ Room و Retrofit خودشان main-safe‌اند.
- `Default` برای کار CPU، `IO` برای I/O مسدودکننده — و dispatcherها را inject کنید.
- `launch` برای side effect، `async` برای نتیجهٔ موازی؛ فرزندِ ناموفق، والد را فوراً لغو می‌کند.
- هر coroutine والد دارد: نه `launch(Job())`، نه `GlobalScope`. کار ماندگار: scope سطح اپ یا WorkManager.
- `lifecycleScope` در DESTROYED می‌میرد — با `repeatOnLifecycle` collect کنید؛ `launchWhenX` منسوخ است.
- لغو مشارکتی است: `ensureActive()`، `runInterruptible`، و همیشه `CancellationException` را دوباره throw کنید.
- `CoroutineExceptionHandler`: فقط روی coroutine ریشه، فقط برای log.
- `coroutineScope` یعنی همه یا هیچ؛ `supervisorScope` یعنی فرزندان مستقل — باز هم داخلشان catch کنید.
- state مشترک: atomicها، `Mutex`، محدودسازی به یک worker — نه `synchronized`. تست: `runTest` و زمان مجازی.

روز ۵ در راه است: Kotlin Flow. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700